DATA SCIENCE / GUIA TÉCNICO EP

Coleções e Laços

Escolha de estruturas, referências, iteração e complexidade

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Listas, tuplas e conjuntos • 2. Mutabilidade, referência e cópia • 3. Iteradores, laços e compreensões • 4. Complexidade e estruturas de busca • 5. Dicionários e contadores • 6. Iteração segura e exaustão • 7. Ordenação e chaves de comparação • 8. Padrões de agregação

Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

01 Listas, tuplas e conjuntos

Listas preservam ordem e permitem alteração; tuplas preservam ordem e não permitem troca de itens; sets eliminam duplicatas e facilitam pertinência. O tipo da estrutura deve refletir a semântica, não apenas a sintaxe.

Exemplo executável

```
nomes = ["Ana", "Bia", "Ana"]
assert len(set(nomes)) == 2
coordenada = (10, 20)
assert coordenada[0] == 10
```

Atenção técnica. Não presuma ordem estável na iteração de set; ordene explicitamente quando precisar comunicar resultados.

Exercício de consolidação. Determine quais operações são apropriadas para lista, tupla e conjunto em um cadastro.

02 Mutabilidade, referência e cópia

Atribuição não cria cópia: dois nomes podem apontar para a mesma lista. list.copy copia somente o nível externo. Estruturas aninhadas exigem análise de aliasing e, às vezes, deepcopy.

Exemplo executável

```
a = [[1], [2]]
b = a.copy()
b[0].append(99)
assert a[0] == [1, 99]

from copy import deepcopy
c = deepcopy(a)
```

Atenção técnica. Nunca use cópia superficial como garantia de isolamento quando os elementos internos são mutáveis.

Exercício de consolidação. Demonstre a diferença entre identidade, igualdade, cópia rasa e cópia profunda.

03 Iteradores, laços e compreensões

for percorre iteráveis; while depende de condição de parada. enumerate evita contadores manuais e zip combina iteráveis em paralelo. Compreensões favorecem transformações simples; geradores economizam memória.

Exemplo executável

```
valores = [4, 7, 8, 10]
pares = [v for v in valores if v % 2 == 0]
assert pares == [4, 8, 10]
for i, valor in enumerate(pares, start=1):
    print(i, valor)
```

Atenção técnica. Um zip convencional interrompe na sequência mais curta; use strict=True quando comprimentos diferentes indicarem erro.

Exercício de consolidação. Converta um laço tradicional em compreensão, mantendo a legibilidade.

04 Complexidade e estruturas de busca

Acesso por índice de lista costuma custar O(1); buscar nela custa O(n). Pertinência em dict e set costuma custar O(1) em média; ordenar custa O(n log n). Tais custos descrevem crescimento, não tempo fixo.

Exemplo executável

```
registros = ["A", "B", "A", "C"]
frequencia = {}
for item in registros:
    frequencia[item] = frequencia.get(item, 0) + 1
assert frequencia["A"] == 2
```

Atenção técnica. Medições dependem do volume de dados e da implementação. Primeiro escolha algoritmo correto, depois avalie gargalos.

Exercício de consolidação. Compare o comportamento de consultas repetidas de pertinência em list e set.

05 Dicionários e contadores

Embora o estudo de funções aprofunde dict, é útil distinguir a necessidade de mapa chave-valor de uma lista de registros. Contagens podem usar get ou collections.Counter. Inserções em dict preservam ordem em Python moderno, mas não representam ordenação semântica dos dados.

Exemplo executável

```
from collections import Counter
itens = ["A", "B", "A", "C"]
assert Counter(itens)["A"] == 2
valores = {"a": 1, "b": 2}
assert list(valores) == ["a", "b"]
```

Atenção técnica. A performance média de hash tables depende de chaves adequadas; valores mutáveis como listas não são hashable.

Exercício de consolidação. Modele um índice por ID usando dict e discuta duplicações.

06 Iteração segura e exaustão

Iteradores podem ser consumidos. Um gerador representa uma computação sob demanda, valiosa para arquivos grandes, mas não é reutilizável sem recriação. Para combinar entradas paralelas, zip(..., strict=True) torna discrepâncias visíveis.

Exemplo executável

```
numeros = (x*x for x in range(4))
assert list(numeros) == [0, 1, 4, 9]
assert list(numeros) == []
assert list(zip([1, 2], ["a", "b"], strict=True)) == [(1,"a"),(2,"b")]
```

Atenção técnica. Conversões list(generator) materializam todos os elementos; evite em fluxos muito volumosos.

Exercício de consolidação. Escreva uma soma com generator expression sem criar lista intermediária.

07 Ordenação e chaves de comparação

sorted cria nova lista; list.sort altera a original. O argumento key define critério de comparação e reverse controla sentido. Ordenação estável preserva a ordem relativa de elementos com a mesma chave.

Exemplo executável

```
dados = [{"nome":"Bia","nota":8}, {"nome":"Ana","nota":8}]
ordenados = sorted(dados, key=lambda d: (d["nota"], d["nome"]))
assert ordenados[0]["nome"] == "Ana"
```

Atenção técnica. Nunca ordene números representados como strings supondo ordem numérica; converta corretamente.

Exercício de consolidação. Ordene transações por valor decrescente e nome crescente.

08 Padrões de agregação

Acumuladores, contadores, agrupamentos e filtros podem ser implementados com laços claros. Para valores numéricos, sum, min, max e any/all frequentemente expressam melhor a intenção do que uma implementação manual, desde que os casos vazios sejam tratados.

Exemplo executável

```
valores = [2, 4, 5, 9]
assert sum(x for x in valores if x % 2 == 0) == 6
assert any(x > 8 for x in valores)
assert all(x >= 0 for x in valores)
```

Atenção técnica. min([]) e max([]) levantam ValueError; avalie default quando valores vazios forem permitidos.

Exercício de consolidação. Crie resumo com total, quantidade e média de valores positivos.

Checklist de revisão

Verifique	Evidência
Conceito	Consegue explicar sem copiar o código?
Execução	O exemplo produz o resultado esperado?
Casos-limite	Entradas inválidas e extremas foram testadas?
Reprodutibilidade	O kernel e as dependências estão registrados?

Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.